Latihan K03 Pak West East

https://docs.google.com/document/d/1rtA9g9F142m7FJ9mVmPKa1rtRhnF_gYvwLMdFW01DZA/edit?usp=sharing

1. Jelaskan dengan singkat apa yang dihitung pada kode ini:

```
int main(int argc, char *argv[]) {
   int rank, received, myvalue, p, neighbor;
   MPI_init(&argc, &argv);
   MPI_Status status;
   MPI_Comm_size(MPI_COMM_WORLD, &p);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   myvalue = rank;
   for(int step = 1; step < p; step = step*2){
      if(rank % step == 0) {
         if(isEven(rank/step)) {
            MPI_Recv(&received, 1, MPI_INT, rank+step, 0, MPI_COMM_WORLD, &status);
            myvalue += received;
         } else {
            break;
         }
      }
      if(rank == 0) {
         printf("%d\n", myvalue);
      }
   }
   return 0;
}
```

Jawaban

Kode ini menghitung **jumlah seluruh rank** (0 + 1 + … + p-1 = p(p-1)/2) dengan reduksi berbentuk pohon (tree-structured reduction). Hasil akhirnya ada di proses 0.

Setiap proses mulai dengan myvalue = rank. Variabel step berlipat dua tiap iterasi (1, 2, 4, …)

- proses yang rank % step == 0 masih "aktif"
- Jika rank/step genap, proses menerima nilai dari rank+step lalu menjumlahkannya
- Jika rank/step ganjil, proses keluar dari loop (break). Seharusnya ia mengirim nilainya ke rank-step, tetapi MPI_Send tidak ada di kode

Contoh p = 8:

- step 1: 0 ← 1, 2 ← 3, 4 ← 5, 6 ← 7
- step 2: 0 ← 2 (berisi 2+3), 4 ← 6 (6+7)
- step 4: 0 ← 4
- Hasil di proses 0: 0+1+...+7 = 28

2. Diberikan kode berikut:

Serial	**Paralel**

for (i = 0; i < n; i++) {
  a[i] = b[i] + 1;
  c[i] = a[i] + 2;
  d[i] = c[i+1] + 1;
}
	#pragma omp parallel for
	for (i = 0; i < n; i++) {
	  a[i] = b[i] + 1;
	  c[i] = a[i] + 2;
	  d[i] = c[i+1] + 1;
	}

Apakah kedua kode tersebut melakukan komputasi yang sama?

Jawaban

Kedua kode tersebut tidak melakukan komputasi yang sama. Pada baris d[i] = c[i+1] + 1, ada loop-carried dependency (anti-dependence/WAR) pada array c.

- **Versi serial**: d[i] membaca c[i+1] sebelum iterasi i+1 menulis c[i+1]. Jadi d[i] selalu memakai nilai **lama** c[i+1]
- **Versi paralel**: iterasi i+1 mungkin dijalankan thread lain dan sudah menulis c[i+1] = a[i+1] + 2 sebelum iterasi i membacanya. Akibatnya, d[i] bisa memakai nilai **baru**. Hasilnya bergantung pada urutan eksekusi (race condition)

Dua baris lainnya aman diparalelkan karena a[i] dan c[i] hanya bergantung pada iterasi yang sama.

Saran perbaikan: Pisahkan menjadi dua loop paralel. Loop pertama menghitung d dari c lama, lalu loop kedua mengubah a dan c. Ada barrier implisit di antara keduanya.

```
#pragma omp parallel for
for (i = 0; i < n; i++) {
  d[i] = c[i+1] +1;
}

#pragma omp parallel for
for (i = 0; i < n; i++) {
  a[i] = b[i] + 1;
  c[i] = a[i] + 2;
}
```

3. Diberikan kode berbasis POSIX thread sebagai berikut (rank adalah id thread). Jika matrix A dan y memiliki tipe elemen `double`, dan cacheline berukuran 64 bytes (8 double), dengan ukuran data 8000 × 8000 ($m = n = 8000$). Jelaskanlah eksekusi kode tersebut pada dual core processor, dengan 4 thread. Apakah mungkin terjadi false sharing jika thread 0 dan thread 1 berjalan pada core yang sama, dan thread 2 dan 3 pada core yang lain? Apakah mungkin terjadi **false sharing** jika thread 0 dan 2 berjalan pada core yang sama, dan thread 1 dan 3 pada core yang lain?

```
void *Pth_mat_vect(void* rank) {
   long my_rank = (long) rank;
   int i, j;
   int local_m = m/thread_count;
   int my_first_row = my_rank*local_m;
   int my_last_row = (my_rank+1)*local_m - 1;

   for (i = my_first_row; i <= my_last_row; i++) {
      y[i] = 0.0;
      for (j = 0; j < n; j++)
         y[i] += A[i][j]*x[j];
   }
   return NULL;
}
```

Jawaban

Eksekusi: m = n = 8000; 4 threads → local_m = 2000:

Thread	**Baris (index y)**	**Cache line ke-**
0	0 - 1999	0 - 249
1	2000 - 3999	250 - 499
2	4000 - 5999	500 - 749
3	6000 - 7999	750 - 999

Tiap thread menulis ke y[i] miliknya sendiri, sedangkan A dan x hanya dibaca. Karena data read-only tidak menyebabkan false sharing, yang perlu dicek hanyalah array y.
Tidak ada cache line yang isinya campuran elemen milik dua thread. Contohnya y[3999] ada di line 499 bersama y[3992..3999] → semuanya milik thread 1. y[4000] ada di line 500 bersama y[4000..4007] → semuanya milik thread 2. Karena tidak ada line yang dibagi, syarat ke-2 tidak terpenuhi dan tidak ada false sharing, bagaimanapun penempatan thread di core.

False sharing terjadi jika dua thread di **core berbeda** menulis variabel berbeda yang berada di **satu cache line** yang sama. Satu cache line berisi 8 double.

Syarat false sharing terjadi:

1. Dua thread berada di **core berbeda** (cache terpisah)
2. Thread-thread menulis variabel berbeda yang berada di cache line yang sama

Satu cache line = 64 byte = 8 double. Jadi y[0..7] satu line, y[8..15] line berikutnya, dan seterusnya, **jika y dimulai tepat di awal cache line** (aligned)

Batas antar-blok y jatuh di indeks 2000, 4000, dan 6000. Ketiganya kelipatan 8 (2000/8 = 250; 4000/8 = 500; 6000/8 = 750). Jika y aligned pada 64 byte, setiap blok dimulai tepat di awal cache line, sehingga **tidak ada cache line yang dibagi dua thread**.

a. Apakah mungkin terjadi false sharing jika thread 0 dan thread 1 berjalan pada core yang sama, dan thread 2 dan 3 pada core yang lain?
   i. Batas 0|1 berada dalam satu core, sehingga tidak mungkin false sharing krn cache-nya sama
   ii. Batas 1|2 (y[3999] dan y[4000]) melintasi core. Jika y aligned, tidak terjadi false sharing
   iii. Batas 2|3 berada dalam satu core.

b. Apakah mungkin terjadi **false sharing** jika thread 0 dan 2 berjalan pada core yang sama, dan thread 1 dan 3 pada core yang lain?
   i. Ketiga batas 0|1; 1|2; 2|3 melintasi core
   ii. Jika y aligned 64 byte, **tetap tidak terjadi false sharing**
   iii. Jika y tidak aligned (misalnya hasil malloc yang hanya aligned 16 byte), satu cache line di tiap batas bisa dipakai bersama. Maka **mungkin terjadi false sharing** di ketiga batas.

4. Diberikan kode MPI berikut yang akan dijalankan oleh 3 proses. Berapakah nilai x, y dan z pada setiap proses di akhir program.

```
int x, y, z;
switch(my_rank) {
   case 0: x=0; y=1; z=2;
      MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);
      MPI_Send(&y, 1, MPI_INT, 2, 43, MPI_COMM_WORLD);
      MPI_Bcast(&z, 1, MPI_INT, 1, MPI_COMM_WORLD);
      break;

   case 1: x=3; y=8; z=5;
      MPI_Bcast(&x, 1, MPI_INT, 0, MPI_COMM_WORLD);
      MPI_Bcast(&y, 1, MPI_INT, 1, MPI_COMM_WORLD);
      break;

   case 2: x=6; y=7; z=8;
      MPI_Bcast(&z, 1, MPI_INT, 0, MPI_COMM_WORLD);
      MPI_Recv(&x, 1, MPI_INT, 0, 43, MPI_COMM_WORLD, &status);
      MPI_Bcast(&y, 1, MPI_INT, 1, MPI_COMM_WORLD);
      break;
}
```

Jawaban

* Operasi kolektif (MPI_Bcast) dicocokkan berdasarkan urutan kemunculannya di setiap proses, bukan berdasarkan nama variabel.

* MPI_Bcast(&variabel, 1, MPI_INT, root, …) artinya siaran
  * root adalah pembicara, isi variabelnya disiarkan
  * semua proses lain harus ikut memanggil MPI_Bcast dengan root yang sama, dan mereka menjadi pendengar. Nilai yang diterima ditulis ke variabel yang mereka tulis di dalam tanda &

Bcast ke-1 (root = 0): P0 mengirim x = 0:
- P0: x tetap 0
- P1: x menjadi 0
- P2: menerimanya ke variabel z, sehingga z menjadi 0

Send/Recv: P0 mengirim y = 1 (tag 43) ke P2, dan P2 menerimanya ke x, sehingga x di P2 menjadi 1.
Bcast ke-2 (root = 1): P1mengirim y = 8:
- P1: y tetap 8
- P0: menerima ke variabel z, sehingga z menjadi 8
- P2: y menjadi 8

Hasil akhir

Proses	**x**	**y**	**z**
0	0	1	8
1	0	8	5
2	1	8	0

5. Dimanakah letak `#omp` yang terbaik pada potongan program di samping?

```
int i, j, k;
float x[n][n][m];
float scale = 2.0;
...
--A--
for(k=1; k<n; k++)
   --B--
   for(j=1; j<n; j++)
      --C--
      for(i=0; i<m; i++)
         x[i][j][k] = x[i][j][k-1] + x[i][j-1][k]*scale
```

Jawaban

di posisi C, yaitu tepat sebelum for(i=0; i<m; i++)
- Loop k punya loop carried dependency, karena iterasi k membaca x[..][..][k-1] yang ditulis iterasi k-1. Tidak boleh diparalelkan, sehingga posisi A salah
- Loop j punya loop-carried dependency, karena membaca x[i][j-1][k] yang ditulis iterasi j-1. Tidak boleh diparalelkan, sehingga posisi B salah

- Loop i tidak punya dependensi, karean setiap iterasi i hanay menyentuh x[i][..][..] miliknya sendiri. jadi hanya loop i yang aman diparalelkan

6. Lengkapi kode CUDA:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

__global__ void Mat_add(float A[], float B[], float C[], int m, int n) {
   int my_id;
   my_id = (1)blockIdx.x * blockDim.x + threadIdx.x;
   C[my_id] = A[my_id] + B[my_id];
}
/* Mat_add */

void Read_matrix(float A[], int m, int n) {
   int i, j;
   for (i = 0; i < m; i++)
      for (j = 0; j < n; j++)
         scanf("%f", &A[i*n+j]);
}
/* Read_matrix */

void Print_matrix(char title[], float A[], int m, int n) {
   int i, j;
   printf("%s\n", title);
   for (i = 0; i < m; i++) {
      for (j = 0; j < n; j++)
         printf("%8.1f", A[i*n+j]);
      printf("\n");
   }
}
/* Print_matrix */
int main(int argc, char* argv[]) {
   int m, n;
   float *h_A, *h_B, *h_C; /* variables in host */
   float *d_A, *d_B, *d_C; /* variables in device */
   size_t size;

   /* Size of matrices */
   m = 512;
   n = 256;
   size = m*n*sizeof(float);

   /* Allocate matrices in host and device memory */
```

```
   (2) h_A = (float*) malloc(size);
   h_B = (float*) malloc(size);
   h_C = (float*) malloc(size);
   cudaMalloc(&d_A, size);
   cudaMalloc(&d_B, size);
   cudaMalloc(&d_C, size);
   Read_matrix(h_A, m, n);
   Read_matrix(h_B, m, n);
   Print_matrix("A =", h_A, m, n);
   Print_matrix("B =", h_B, m, n);

   /* Copy matrices from host memory to device memory */
   (3) cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
   cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

   /* Invoke kernel using m thread blocks, each contains n threads */
   (4) Mat_add<<<m, n>>>(d_A, d_B, d_C, m, n);

   /* Wait for the kernel to complete */
   (5) cudaDeviceSynchronize();

   /* Copy result from device memory to host memory */
   (6) cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

   Print_matrix("The sum is:", h_C, m, n);

   /* Free host & device memory */
   (7) free(h_A);
   free(h_B);
   free(h_C);
   cudaFree(d_A);
   cudaFree(d_B);
   cudaFree(d_C);

   return 0;
}
/* main */
```

CUDA explained
CUDA adalah cara memakai GPU untuk menghitung. Ada dua tempat dengan memori terpisah: - **Host = CPU + RAM biasa**. Variabelnya diberi awalan h_ - **Device = GPU + memori GPU**. Variabelnya diberi awalan d_

Kita punya dua set variabel karena ada dua memori yang berbeda

Aturan emas: GPU tidak bisa membaca memori CPU secara langsung, dan sebaliknya. Maka alurnya selalu:

1. Siapkan data di host
2. Salin data ke device
3. GPU menghitung
4. Salin hasil kembali ke host
5. Bebaskan memori

Thread di CUDA disusun berkelompok:

- Block = kelompok thread
- Grid = kumpulan semua block

Tiga variabel bawaan CUDA:

Variabel	Artinya
`threadIdx.x`	Nomor thread di dalam block-nya (mulai 0)
`blockIdx.x`	Nomor block-nya (mulai 0)
`blockDim.x`	Jumlah thread per block

Analogi gedung apartemen: block = lantai, thread = nomor kamar di lantai itu. Jika setiap lantai memiliki 256 kamar, maka nomor kamar global:
nomor lantai x jumlah kamar per lantai + nomor kamar di lantai itu

Thats why (1) rumusnya:
my_id = blockIdx.x * blockDim.x + threadIdx.x;

Di host pakai malloc biasa (fungsi C standar), sedangkan di device pakai cudaMalloc (fungsi CUDA khusus untuk meminta memori di GPU) → &d_A karena cudaMalloc perlu mengubah isi pointer d_A supaya menunjuk ke alamat memori GPU.

Format cudaMemcpy(tujuan, sumber, jumlah_byte, arah)
arah: either cudaMemcpyHostToDevice atau cudaMemcpyDeviceToHost
Hanya A dan B yang disalin pada (3) karena mereka input sedangkan C akan diisi oleh GPU

Format pemanggilan kernel:
namaKernel <<< jumlah_block, thread_per_block>>>(argumen…);
argumen yang dikirim adalah pointer device.

ingat:

- free untuk memori hasil malloc
- cudaFree untuk hasil cudaMalloc